

Spatial Analysis and Modeling

Machine Learning for Spatial Data



UNIVERSITÀ
DI TORINO

Why this module?

So far we modelled space to **explain** — Moran's I, spatial econometrics, GWR, kriging. Interpretable coefficients, statistical inference.

Now we switch goal to **predict**, and meet machine learning as a first-class predictor. The catch: spatial data breaks ML's core i.i.d. assumption, so we must adapt both the *models* and the *validation*.

- **A — foundations & the spatial catch:** a first model, then why standard cross-validation **leaks** on spatial data.
- **B — spatially-aware ML:** does Random Forest match kriging?
- **C — graph neural networks:** spatial data *is* a graph.
- **D — representation learning:** location encoding & GeoAI foundation models.

One thread ties it together: **Tobler's law** — near things are similar — is both the problem and the tool.

A — Machine learning, first

Supervised learning — predict vs. explain

The econometric spine asked *why*: estimate β , test hypotheses. Machine learning asks *how well can we predict*: fit a flexible function, measure it on held-out data.

- **Ingredients**: features $\mathbf{X}$ $\rightarrow$ target $\mathbf{y}$; a model with **fit** and **predict**.
- **Regression** (continuous $\mathbf{y}$) vs **classification** (categorical $\mathbf{y}$).
- **The deal**: give up clean coefficients, gain non-linear, high-dimensional fits.
- Success is **out-of-sample accuracy**, not in-sample $\mathbf{R}^2$ or **p**-values.

The rest of this module is a tour of models — and the discipline to trust them.

A first model — Random Forest on San Diego Airbnb

We predict **log price** from listing attributes + coordinates, on the San Diego Airbnb data from *Geographic Data Science with Python* (already in **data/airbnb/**).

```
feat = ["accommodates", "bedrooms", "beds", "bathrooms",  
        "guests_included", "reviews_per_month", "number_of_reviews",  
        "host_listings_count", "longitude", "latitude"]  
X = df[feat].fillna(0.0).values  
y = np.log1p(df["price"].values)  
  
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25)  
rf = RandomForestRegressor(n_estimators=300).fit(X_tr, y_tr)  
# train R^2 = 0.955    test R^2 = 0.667    (random split)
```

Notebook: notebooks/08_spatial_ml_A_foundations_crossval.ipynb

The forest fits the training data almost perfectly, generalises to **0.667** on the held-out quarter. That test score looks honest — on spatial data, it is **not**.

Lesson — a strong random-split score can be a spatial illusion; never trust it before validating spatially.

Over-fitting, and an honest-*looking* split

The gap between **train $R^2 = 0.955$** and **test $R^2 = 0.667$** is **over-fitting**: the forest memorises training rows, then generalises less well.

- The random train/test split is the *standard* ML honesty check.
- It controls for memorisation — but assumes train and test are **independent**.
- On spatial data that assumption fails, and even the test score flatters us.

Why? That is the spatial catch.

A — The spatial catch: leakage

Tobler's law → data leakage

"Near things are more related than distant things."

A **random** split scatters neighbours across train and test. For almost every test listing there is a near-duplicate next door in the training set.

- The model **interpolates between near-duplicates** instead of generalising.
- It "peeks" across the split — this is **spatial data leakage**.
- Result: the test score measures interpolation, not prediction in new places.

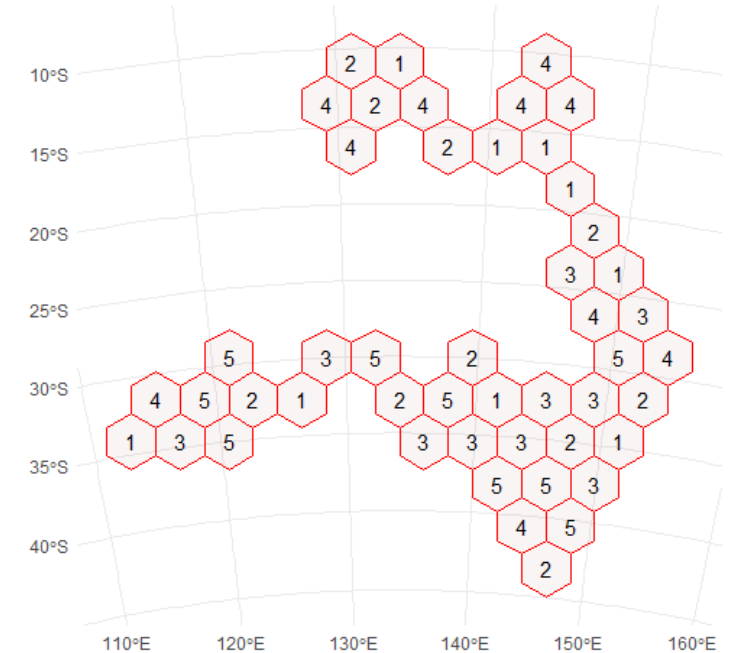
The fix is to make the split respect geography.

Spatial cross-validation

Hold out **whole regions**, not random points, so train and test are spatially separated. Sun, Hu et al. (2023) organise the methods into four families:

- **Clustering-based:** k-means the coordinates, hold out clusters.
- **Grid / block:** tile space, hold out tiles.
- **Geo-attribute:** split by an existing spatial grouping.
- **Buffered leave-one-out:** drop a guard ring around each test point.

The choice encodes *which* prediction task you care about.



Worked example — random vs. spatial CV

Same Random Forest, same San Diego data, two validation schemes: random **KFold** vs. **GroupKFold** over 10 geographic clusters.

```
groups = KMeans(n_clusters=10).fit_predict(df[["longitude", "latitude"]])
r2_random = cross_val_score(rf, X, y, cv=KFold(10, shuffle=True))
r2_spatial = cross_val_score(rf, X, y, groups=groups, cv=GroupKFold(10))
# random k-fold R^2 = 0.664 +/- 0.043
# spatial k-fold R^2 = 0.539 +/- 0.131
# optimism (random - spatial) = 0.125 R^2 points
```

The random score is **0.125 R² points too high**, with far larger fold-to-fold variance. That gap is the leakage — performance you will *not* get in new areas.

Notebook: notebooks/08_spatial_ml_A_foundations_crossval.ipynb

Lesson — report the spatial-CV score; the random one is the number you wish were true.

The certified case — Ploton et al. 2020

Ploton et al. (*Nature Communications* 2020) quantified this for a Random Forest mapping **tropical-forest biomass** — the canonical warning in the field.

Validation	Reported R^2
Random 10-fold CV	0.53
Spatial 44-fold CV (150 km)	0.14
Null (intercept-only)	≈ 0

Buffered leave-one-out: R^2 decays from ~ 0.50 at 0 km to ~ 0.15 by 50 km, $\approx$ null beyond 100 km. *A model that "explained half the variance" had near-zero real skill.*

Lesson — a published R^2 means little without spatial validation — here it fell from 0.53 to near zero.

...but teach the debate

Spatial CV is best practice, **not settled dogma**.

- **Wadoux et al. (2021, *Ecological Modelling*)**: spatial CV can be *over-*pessimistic for map-accuracy assessment when samples are representative.
- The honest answer depends on the **question**: interpolate *within* sampled areas, or extrapolate to *new* regions?

Rule of thumb: report **both** random and spatial scores, and state which prediction task you are answering. Every later module is validated this way.

B — Spatially-aware ML

Plain Random Forest ignores space

Give a forest raw **(x, y)** and it splits space into **axis-aligned boxes** — no notion of smooth distance decay. The result is the **salt-and-pepper** artefact we saw in **03b_spatial_saltpepper_basic**: noisy, blocky maps.

- Kriging (deck 02b) instead uses the **variogram**: how similarity decays with distance. It is the geostatistics gold standard for interpolation.
- Can a Random Forest be taught the same distance structure?

Notebook: [notebooks/03b_spatial_saltpepper_basic.ipynb](#)

RFsp — Random Forest for spatial predictions

Hengl et al. (2018, *PeerJ*): add, as features, the **buffer distances** from each point to every observation. The forest can then learn distance decay — the role spatial autocorrelation plays in kriging.

$$\mathbf{X}_{\text{RFsp}}(s) = \left[d(s, s_1), d(s, s_2), \dots, d(s, s_n) \right]$$

- Per fold: training features = pairwise distance matrix among training points; test features = distances to those **same** points (columns match, so it transfers).
- Headline claim: *RFsp predicts as accurately as ordinary kriging.*

Worked example — Meuse zinc, location only

The canonical **Meuse** heavy-metals dataset (`data/meuse/meuse.csv`), `log(zinc)`, 5-fold CV. Location only, to compare fairly with kriging.

```
# same 5-fold CV, refit each fold
OK (kriging)           CV-RMSE(log zinc) = 0.424 +/- 0.079
plain RF (x, y)        CV-RMSE(log zinc) = 0.502 +/- 0.103
RFsp (buffer dist)     CV-RMSE(log zinc) = 0.457 +/- 0.107
```

Coordinate-only RF is worst (the box artefact). **Buffer distances recover most of the gap toward kriging** — Hengl's result. On this tiny, smooth set kriging still edges ahead (the paper's own caveat: small + linear favours kriging).

Notebook: `notebooks/09_spatial_ml_B_rfsp_kriging.ipynb`

Lesson — feed a forest distances, not raw coordinates, and ML rivals kriging at pure interpolation.

Where ML pulls ahead — covariates

Kriging uses only location. A forest folds in **covariates** (**dist** to river, **elev**) *and* space, in one model.

OK (location only)	CV-RMSE(log zinc) = 0.424
RF (x, y + covars)	CV-RMSE(log zinc) = 0.299
RFsp (+ covars)	CV-RMSE(log zinc) = 0.303

With covariates both forests now **beat** ordinary kriging (**0.30** vs **0.42**) — kriging needs extra machinery (regression kriging) to use them. That is the practical case for spatially-aware ML: structure **and** covariates together.

Notebook: notebooks/09_spatial_ml_B_rfsp_kriging.ipynb

Lesson — ML's real edge over kriging is fusing covariates *and* space in a single model.

The wider toolbox

- Gradient boosting (**XGBoost**, **LightGBM**) plugs in the same way — often the strongest tabular predictor.
- Feed it the spatial features from **03b_features_engineering** : spatial lag, coordinates, **Moran eigenvectors**, embeddings.
- **Kopczewska (2022, *Ann. Reg. Sci.*)** frames the whole move as ML *complementing* spatial econometrics — clustering (unsupervised) + prediction (supervised) — not replacing it.
- All of it stays honest only under **spatial CV** (module A).

C — Graphs & GNNs

Spatial data *is* a graph

A PySAL spatial-weights matrix **W** (deck 05) is an **adjacency matrix**.

- **Nodes** = areas / points; **edges** = neighbours (Queen, KNN, distance band).
- A **graph neural network** consumes exactly this: features on nodes + edges.
- We already build **W** all the time — a GNN is what learns on top of it.

$$W \equiv \text{edge_index} \quad (\text{neighbours} \rightarrow \text{graph})$$

A GCN layer = learnable spatial lag

One **graph convolution** (Kipf & Welling, 2017) updates each node from its neighbours:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right)$$

- $\tilde{A}$ = adjacency + self-loops: **average the neighbourhood**, then a learned transform **W** and non-linearity **σ** .
- That neighbourhood average is a **spatial lag** — but learned, non-linear, stacked.
- Two layers = information from **two hops** away.

Cora — when structure carries the signal

The canonical first-GNN benchmark (2,708 papers, citation edges, 7 classes), loaded from the course's own **data/Cora/**. GCN vs. an MLP on the *same features* that ignores the graph.

```
ds = Planetoid(root="data", name="Cora")
# MLP (features only, ignores graph)  test accuracy = 0.542
# GCN (uses the citation graph)      test accuracy = 0.806
```

The graph adds **+26 accuracy points**: on Cora the *structure* carries information the node features miss. This is the result that launched modern GNNs.

Notebook: notebooks/10_spatial_ml_C_gnn.ipynb

Lesson — reach for a GNN when the graph itself carries signal your node features miss.

Airbnb as a graph — message passing smooths

Build a **KNN(8)** graph on the San Diego coordinates (module A's data), predict **log price** with a GCN vs. an MLP, and measure **Moran's I of the predictions**.

```
w = KNN.from_array(coords, k=8)    # deck-05 weights -> edge_index
# actual log-price    Moran's I = 0.191
# MLP    test R^2 = 0.595    prediction Moran's I = 0.093
# GCN    test R^2 = 0.135    prediction Moran's I = 0.911
```

Here the GCN is **less accurate** but its map is far **smoother** (Moran **0.91**):
message passing **over-smooths** when node features already carry the signal.

Notebook: notebooks/10_spatial_ml_C_gnn.ipynb

Lesson — a GNN is not free: it imposes smoothing, so it can *hurt* when features already dominate.

GNNs are not a free lunch

Cora: structure wins. Airbnb: features win, the graph over-smooths. The skill is knowing which regime you are in.

- GNNs **impose** spatial structure on predictions — sometimes helpful, sometimes blur.
- **De Sabbata & Liu (2023, *IJGIS*)**: a graph autoencoder for **London geodemographics** gives class homogeneity similar to classic methods but **higher spatial clustering** — exactly the smoothing we measured. Open code, reuses London/LSOA census data.
- Use a GNN when **neighbourhood structure carries signal** your features miss.
- To make a GNN *properly* spatial, inject a location encoder → **module D**.

D — Representation learning

Raw coordinates are bad inputs

Two nearby places can have very different **(lat, lon)** numbers, and the map from coordinate to target is not smooth. Neural nets struggle with raw coordinates.

A **location encoder** **$\text{Enc}(\mathbf{x}) \rightarrow \mathbb{R}^d$** turns a coordinate into a smooth, multi-scale feature vector — the geospatial analogue of a Transformer's **positional encoding**.

$$\text{Enc}(s) = \left[\sin(2^k \pi s), \cos(2^k \pi s) \right]_{k=0}^{L-1}$$

Low frequencies = broad trends; high frequencies = local detail (**Space2Vec**, Mai et al., ICLR 2020).

Worked example — location alone, California housing

The encoder is ~5 lines. Predict house value from **location only**, same MLP on raw coordinates vs. encoded coordinates.

```
def location_encoder(coords, L=10):  
    out = []  
    for i in range(L):  
        f = 2.0 ** i * np.pi  
        out += [np.sin(f * coords), np.cos(f * coords)]  
    return np.concatenate(out, axis=1)      # 2 dims -> 40 dims  
# raw (lat, lon)      MLP test R^2 = 0.435  
# location-encoded    MLP test R^2 = 0.745
```

Same model, same coordinates — encoding lifts location-only R^2 from **0.44** to **0.75**. Encode location; do not feed raw coordinates.

Notebook: notebooks/11_spatial_ml_D_location_encoding.ipynb

Lesson — encode coordinates (Fourier / grid-cell) before a neural net — raw lat/lon are poor inputs.

The location-encoding zoo

Our 5 lines are the seed of a fast-moving area (Mai et al. 2022 review):

- **Space2Vec** — multi-scale grid-cell encoder (the method above).
- **SatCLIP** (Klemmer 2025) — *global* encoder, CLIP-trained on Sentinel-2; pretrained checkpoints + notebooks.
- **GeoCLIP** — image 
GPS alignment; reusable GPS encoder (`pip install geoclip`).
- **CSP** — contrastive spatial pre-training; boosts species ID from few labels.
- **PE-GNN** (Klemmer 2023) — location encoder + **GNN** (module C) that **matches Gaussian Processes** for spatial regression — closing the loop back to kriging.

Earth-observation foundation models

Awareness — GPU + big data, not a laptop lab. A **geospatial foundation model** is pretrained self-supervised on massive satellite imagery, then **fine-tuned** on a small labelled task (flood, crop, burn scar).

- **Two paradigms:** masked autoencoding (**Prithvi**, **SatMAE**, **Presto**) vs. contrastive (**SatCLIP**, **CSP**).
- **Prithvi-EO-2.0** (IBM/NASA) — multi-temporal ViT on HLS imagery.
- **Presto** — a deliberately *tiny*, laptop-friendly RS foundation model.
- **Tooling:** **TerraTorch** (fine-tuning) on top of **TorchGeo** (datasets, samplers, pretrained weights, CRS-aware tiling).

When to use what

Situation	Reach for
Need interpretable coefficients, inference	Classical (GWR, spreg)
Strong tabular covariates, prediction	Gradient boosting / RF (+ spatial features)
Pure interpolation from location	Kriging or RFsp
Structure/relations carry signal	GNN
Location as a feature for a neural net	Location encoder
Satellite imagery, little labelled data	EO foundation model + fine-tune

Always: validate spatially (module A). Prediction $\neq$ explanation — pick the tool for the question.

The landscape — where this is taught

Graduate GeoAI / spatial-ML courses to explore further:

- **Penn State** GEOG 582 *AI in Spatial Data Science*; **UGA** GEOG 4921/6921 *Advanced Geospatial AI* (Mai).
- **Stanford** CS325B *Data for Sustainable Development* (Burke/Ermon/Lobell); **ETH Zürich** EcoVision (Schindler/Wegner).
- **ITU Copenhagen** *Geospatial Data Science* (Szell); **Liverpool** GDSSL ENVS363/563 (Arribas-Bel) — closest to this course's Python/PySAL stack.
- **TU Munich** EO-College *ML for Earth Observation* (Zhu); **UF** GEO 6938 *GeoAI*.

References

Spatial cross-validation & leakage

- **Ploton, P., et al. (2020).** Spatial validation reveals poor predictive performance of large-scale ecological mapping models. *Nature Communications*, 11, 4540.
- **Roberts, D. R., et al. (2017).** Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913-929.
- **Sun, Z., Hu, Y., et al. (2023).** Spatial cross-validation. In *Handbook of Geospatial Artificial Intelligence*. CRC Press.
- **Wadoux, A. M. J.-C., et al. (2021).** Spatial cross-validation is not the right way to evaluate map accuracy. *Ecological Modelling*, 457, 109692.

References

Spatially-aware ML & graph neural networks

- **Hengl, T., et al. (2018).** Random forest as a generic framework for predictive modeling of spatial and spatio-temporal variables. *PeerJ*, 6, e5518.
- **Kopczewska, K. (2022).** Spatial machine learning: new opportunities for regional science. *Annals of Regional Science*, 68(3), 713-755.
- **Kipf, T. N., & Welling, M. (2017).** Semi-supervised classification with graph convolutional networks. *ICLR*.
- **De Sabbata, S., & Liu, P. (2023).** A graph neural network framework for spatial geodemographic classification. *IJGIS*, 37(12), 2464-2486.

References

Location encoding & foundation models

- **Mai, G., et al. (2022).** A review of location encoding for GeoAI: methods and applications. *IJGIS*, 36(4), 639-673.
- **Mai, G., et al. (2020).** Multi-scale representation learning for spatial feature distributions using grid cells (Space2Vec). *ICLR*.
- **Klemmer, K., et al. (2023).** Positional encoder graph neural networks for geographic data. *AISTATS*.
- **Klemmer, K., et al. (2025).** SatCLIP: global, general-purpose location embeddings with satellite imagery. *AAAI*.

References

Software & datasets

Python Libraries

- **scikit-learn** — Random Forest, cross-validation: <https://scikit-learn.org/>
- **PyTorch Geometric** — GNNs (GCN, PE-GNN): <https://pyg.org/>
- **gstools** — variograms & kriging: <https://github.com/GeoStat-Framework/GSTools>
- **TorchGeo / TerraTorch** — EO foundation models: <https://github.com/microsoft/torchgeo>

Datasets used

- **San Diego Airbnb** — *Geographic Data Science with Python*: <https://geographicdata.science/>
- **Meuse heavy metals** — classic geostatistics dataset (R `sp` / `gstat`).
- **Cora citation network** — GNN benchmark (Kipf & Welling), via PyG `Planetoid` .
- **California housing** — `sklearn.datasets.fetch_california_housing` .